

Natürliche Sprachverarbeitung mit KI

Erik Voigt, Johannes Langer

Gliederung



01 Natürliche Sprachverarbeitung: Grundlagen

02 Probabilistische Sprachmodelle
- 2.1 Definition und Begriffe
- 2.2 N-Gram Modelle
- 2.3 Skalierbarkeit & Curse of Dimensionality

03 Vektorrepräsentationen

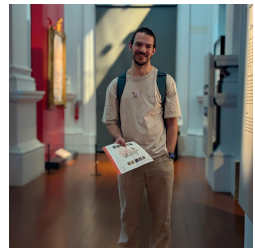
04 Neuronale Sprachmodelle
- 4.1 Attention
- 4.2 Autoregression
- 4.3 Dialogsysteme

05 Abschluss

Über Uns



erik.voigt@uni-bamberg.de



johannes.langer@uni-bamberg.de

Wissenschaftliche Mitarbeiter am Lehrstuhl Kognitive Systeme der Universität Bamberg, Forschung und Lehre im Bereich KI und maschinelles Lernen



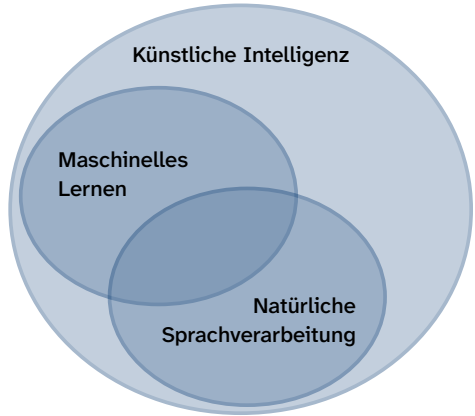
§1

Natürliche Sprachverarbeitung: Grundlagen



Natürliche Sprachverarbeitung

- Teilbereich der Künstlichen Intelligenz
  - Wie können Computer mit natürlicher (d.h. menschlicher) Sprache umgehen?
  - Wie können relevante Informationen aus Texten extrahiert und verarbeitet werden?
- Wichtigste Herausforderung: Ambiguität, z.B.
  - syntaktisch
  - lexikalisch
  - kontextuell
  - ...



Natürliche Sprachverarbeitung

Historischer Ansatz: Eliza (Weizenbaum, 1966)

- Textbasierte Simulation eines Roger'schen Psychotherapeuten
- Wird als erster „Chatbot" bezeichnet
- Einfaches Programm, dass Regeln und Vorlagen nutzt, die von Expert:innen entworfen wurden
- Die Eingabe wird in Schlüsselwörter zerlegt, die bestimmte Regeln auslösen
- Die Antworten werden aus ebendiesen, bzw. verknüpften Regeln generiert
- Demo: https://www.masswerk.at/eliza/

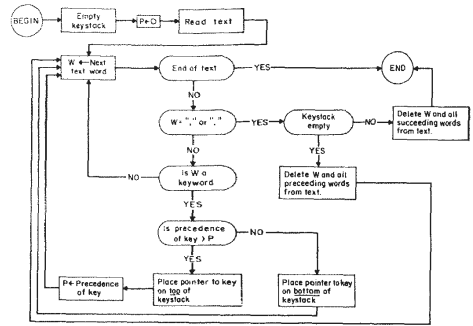


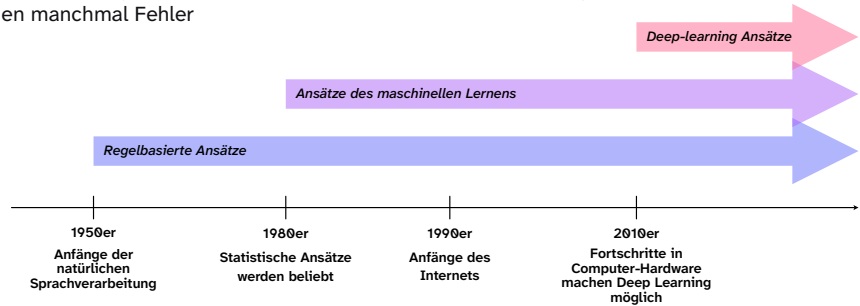
FIG. 2. Basic flow diagram of keyword detection

Weizenbaum, J. (1966). ELIZA—a Computer Program for the Study of Natural Language Communication between Man and Machine. *Communications of the ACM, 9*(1), 36–45. https://doi.org/10.1145/365153.365168

Natürliche Sprachverarbeitung

Zeitstrahl

- Regelbasiert: Brauchen keine Daten, nutzen Expertenwissen, können Verhalten garantieren
- Statistisch: Benötigen Daten, sind dort anwendbar, wo komplexe Regeln schwer zu formulieren/finden sind, machen manchmal Fehler



Kochmar, E. (2022, November). *Getting Started with Natural Language Processing.* Simon and Schuster

Maschinelles Lernen

Definition

DEF 1.1 · Maschinelles Lernen

Ein Computerprogramm **lernt** aus Erfahrung E im Bezug auf Aufgaben T und Leistungsmaß P, wenn dessen Leistung in Aufgaben T, gemessen durch P, mit Erfahrung E steigt (Mitchell, 1997).

Erfahrung	Daten repräsentiert als Mengen von Attributen (z.B. demografische Daten und gesundheitliche Geschichte); Bilder; Spielzüge; Beobachtungen (z.B. Folge von Ereignissen, oder auch Wörtern...)
Aufgabe	Beispielsweise Klassifikation (hohes/mittleres/geringes Krankheitsrisiko); Objekterkennung; Schach spielen; nächstes Ereignis/Wort vorhersagen
Leistungsmaß	Üblicherweise eine Form von *Genauigkeit* (Prozentsatz der korrekten Schätzungen auf *unbekannten* Daten), aber auch Interpretierbarkeit und *Anwendbarkeit* für Menschen

Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill

Maschinelles Lernen



Anwendungsfelder

- Großer praktischer Nutzen in vielen verschiedenen Bereichen
- Daten-Mining: Große Datenbanken enthalten häufig wertvolle implizite Regelmäßigkeiten/Muster, die automatisch entdeckt werden können (Ergebnisse von medizinischen Behandlungen, Vorzüge von Kund:innen)
- Komplexe Domänen, wo Menschen das nötige Wissen entweder nicht haben, oder nicht formulieren können, um effiziente Algorithmen zu entwickeln (z.B. Gesichtserkennung auf Bildern)
- Domänen, wo sich das Programm dynamisch an sich ändernde Bedingungen anpassen muss (z.B. Fertigungsprozesse unter sich ändernden Materialvorräten kontrollieren)
- Autonome Systeme (Roboter, selbstfahrende Autos)

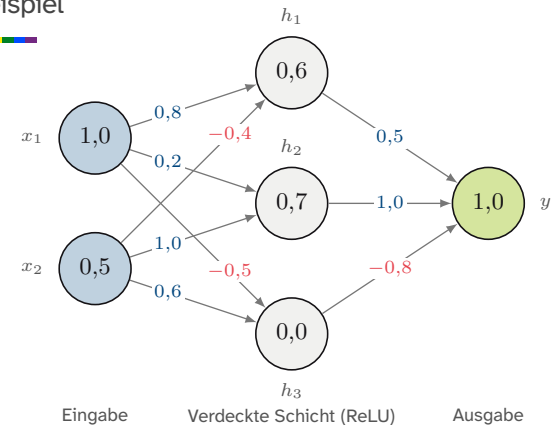
Neuronale Netze



- Neuronale Netze sind nur einer von vielen verschiedenen Algorithmen für maschinelles Lernen
- Sie sind differenzierbare Funktionen mit (vielen) variablen (*lernbaren*) Parametern, die Anhand einer Fehlerfunktion (siehe *Leistungsmaß* der Definition) *optimiert* werden.
- Sie bestehen häufig aus mehreren *Schichten*, wobei eine Schicht einer differenzierbaren Teilfunktion entspricht ($\rightarrow$ Kettenregel)
- Neuronale Netze arbeiten vollständig auf reellwertigen Ein- und Ausgaben
- Viele Schichten lassen sich als Matrixmultiplikationen repräsentieren, wodurch sie auf Grafikprozessoren sehr schnell berechnet werden können

Neuronale Netze

Beispiel



Jedes Neuron berechnet die gewichtete Summe seiner Eingaben und wendet eine *Aktivierungsfunktion* an, hier $\mathrm{ReLU}(z) = \max(0, z)$:

$$h_1 = \max(0,\ 0{,}8 \cdot 1{,}0 - 0{,}4 \cdot 0{,}5) = 0{,}6$$
$$h_2 = \max(0,\ 0{,}2 \cdot 1{,}0 + 1{,}0 \cdot 0{,}5) = 0{,}7$$
$$h_3 = \max(0,\ -0{,}5 \cdot 1{,}0 + 0{,}6 \cdot 0{,}5) = 0$$
$$y = 0{,}5 \cdot 0{,}6 + 1{,}0 \cdot 0{,}7 - 0{,}8 \cdot 0 = 1{,}0$$

Die Gewichte sind die *lernbaren Parameter*: Beim Training werden sie so angepasst, dass y dem gewünschten Wert näher kommt.



§2

Probabilistische Sprachmodelle

Probabilistische Sprachmodelle

Definition

DEF 2.1 · Probabilistisches Sprachmodell

Ein **statistisches** oder **probabilistisches Sprachmodell** kann durch die *bedingte Wahrscheinlichkeit* des nächsten Wortes, gegeben aller vorherigen Worte repräsentiert werden. Dann ist die Wahrscheinlichkeit einer Wortfolge

$$\hat{P}(w_1, \ldots, w_T) = \prod_{i=1}^{T} \hat{P}(w_i \mid w_1, \ldots, w_{i-1}).$$

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, *3*, 1137–1155

Begriffe

DEF 2.2 · Token

Ein **Token** ist die kleinste diskrete Einheit in der Sprachmodellierung. Token können Zeichen, Wortteile, Wörter, oder sogar ganze Sätze sein.

DEF 2.3 · Vokabular

Das **Vokabular** V ist die Menge der einzigartigen *Token* bei der Sprachmodellierung. Die Größe des Vokabulars $|V|$ beschreibt den *Zustandsraum* des Modells. Token, die nicht im Vokabular enthalten sind, kennt das Modell nicht, und können nicht verarbeitet werden.

N-Gram Modelle



In der Praxis ist die Berechnung beliebig langer Token-Sequenzen unmöglich. Ein *N-Gram Sprachmodell* trifft die *Markov-Annahme* N-ter Ordnung: Die Wahrscheinlichkeit des Tokens i hängt nur von den vorherigen $N-1$ Token ab. N wird auch als *Ordnung* des N-Gram Modells bezeichnet.

DEF 2.4 · N-Gram Sprachmodell

Ein **N-Gram Sprachmodell** ist ein *probabilistisches Sprachmodell* unter der Markov-Annahme N-ter Ordnung:

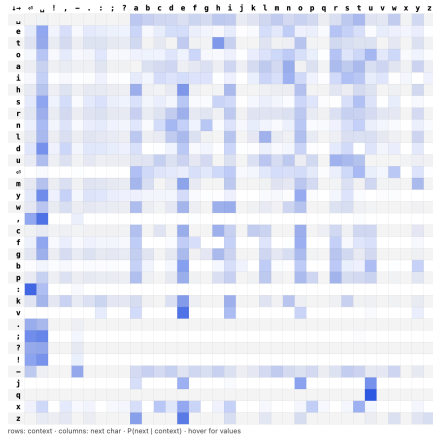
$$\hat{P}(w_i \mid w_1, \ldots, w_{i-1}) \approx \hat{P}(w_i \mid w_{i-(N-1)}, \ldots, w_{i-1})$$

Veritasium. (2025). The Strange Math That Predicts (Almost) Anything.

N-Gram Modelle

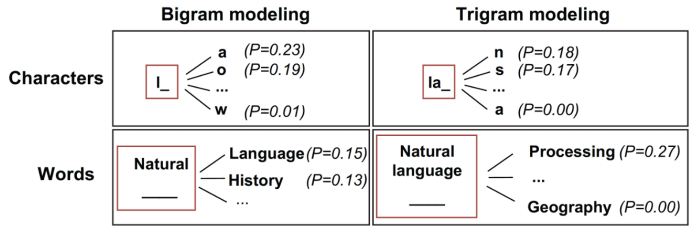
Training

Gesetz der großen Zahlen: Das Training des Modells veranschaulicht das empirische Gesetz der großen Zahlen: Wir schätzen die wahre, unsichtbare Wahrscheinlichkeit eines Folgetokens, indem wir einfach seine *relative Häufigkeit* in einem großen Textkorpus ermitteln.



N-Gram Modelle

Sprachgenerierung



Sampling: Das Generieren des nächsten Tokens ist ein Zufallsexperiment. Im Gegensatz zu einem fairen Würfel (Gleichverteilung) ziehen wir hier aus einer *gewichteten Verteilung*. Das Modell „würfelt“ das nächste Zeichen, wobei Zeichen mit hoher relativer Häufigkeit eine entsprechend größere Trefferfläche auf unserem gedachten „Sprach-Würfel“ haben.

Kochmar, E. (2022, November). *Getting Started with Natural Language Processing*. Simon and Schuster

Curse of Dimensionality

DEF 2.5 · Curse of Dimensionality

Der **Curse of Dimensionality** veranschaulicht exponentielles Wachstum. Bei einem Vokabular der Größe $|V|$ gibt es für ein N-Gram Modell exakt $|V|^N$ mögliche Kombinationen (Ziehen mit Zurücklegen unter Beachtung der Reihenfolge).

Kombinatorik · Exponentialfunktion

BSP 2.6 · Curse of Dimensionality

Für ein einfaches Bi-Gram auf Charakterebene ohne Sonderzeichen und Groß- und Kleinschreibung ergibt sich eine Matrix der Größe:

$$|V|^N = 26^2 = 676$$

Für ein modernes Sprachmodell wie GPT-4 liegt die Größe des Vokabulars bei 200.000. Bereits bei einem Bi-Gram würde sich damit eine Matrix der Größe

$$|V|^N = 200.000^2 = 4 \times 10^{10}$$

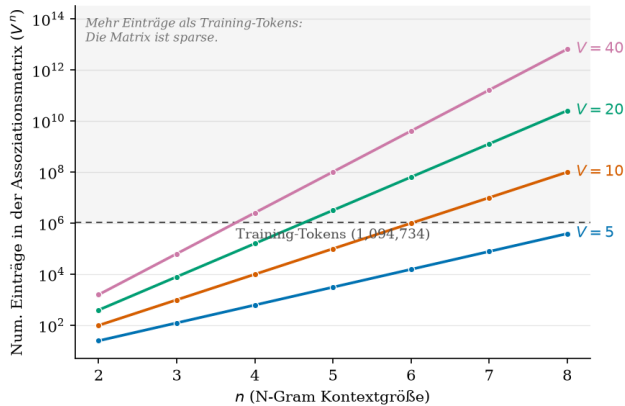
ergeben. Diese Matrizen sind *schwachbesetzt*.

Curse of Dimensionality

Spätestens ab der gestrichelten Linie treten *theoretische* Sequenzen auf, die in den Daten nicht repräsentiert sind. Diese Kombinationen haben eine Übergangswahrscheinlichkeit von 0; durch die multiplikative Verknüpfung ist dann die Wahrscheinlichkeit der gesamten Folge 0!

Durch *Laplace-Smoothing* werden kleine konstante Werte zu jeder möglichen Zelle hinzugefügt, um den vollständigen Verfall der Wahrscheinlichkeiten zu verhindern.

$$P(r \mid q) = \frac{\texttt{Alte Anzahl (q} \rightarrow \texttt{r)} + 1}{\texttt{Alte Zeilensumme q} + 26}$$



Gedächtnisverlust



Ein N-Gram Modell hat ein Kontextfenster (Gedächtnis) von $N - 1$. Eine Frage wie:

„John und Mary sitzen am Tisch. Mit wem sitzt John am Tisch? …“

würden Sie vermutlich problemlos mit „Mary“ beantworten. Ein N-Gram Modell geringer Ordnung hat an der Stelle jedoch leider wieder „vergessen“, dass es Mary jemals gab. N-Gram Sprachmodelle sind also nicht in der Lage, Informationen über längere Kontexte zu erhalten.



§3

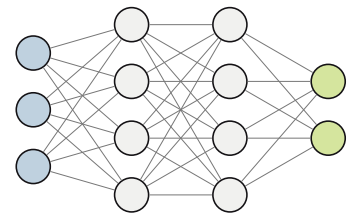
Vektorrepräsentationen

Motivation

- Neuronale Netze verwenden dieselben Parameter, unabhängig vom Input
- Die Wahrscheinlichkeit einer Sequenz würde also durch die kombinierte Aktivierung verschiedener Parameter repräsentiert werden, anstelle durch eine exklusive Zelle in einer großen Matrix

→ Das löst unser Skalierungsproblem (teilweise)

- Aber: Neuronale Netze brauchen reellwertige Eingaben, d.h. die *diskreten* Token müssen *vektorisiert* werden



Embeddings

One-Hot Vektoren

Idee: Die Vektorrepräsentation hat die Größe $|V|$, wobei jedes Element im Vektor einem möglichen Token entspricht. Um ein Token zu repräsentieren, wird das entsprechende Element auf 1 gesetzt; alle anderen Elemente bleiben 0.

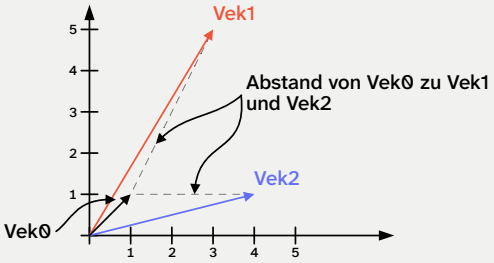
- Simpel
- Hochdimensional (erinnere GPT-4, $|V| = 200.000$)
- Per definition schwachbesetzt $\rightarrow$ ineffizient
- keine Semantik

Abstandsmaße

Euklidische Distanz

$$d(p,q) = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$$

- Für $n = 2$ entspricht es dem Satz des Pythagoras
- Wenig aussagekräftig in höheren Dimensionen

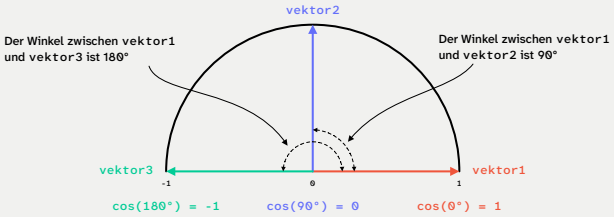


Abstandsmaße

Kosinusähnlichkeit

$$S_C(p,q) = \frac{p \cdot q}{\|p\| \, \|q\|}$$

- Entspricht dem Winkel zwischen zwei Vektoren
- Robust in höheren Dimensionen



Embeddings

Word2Vec

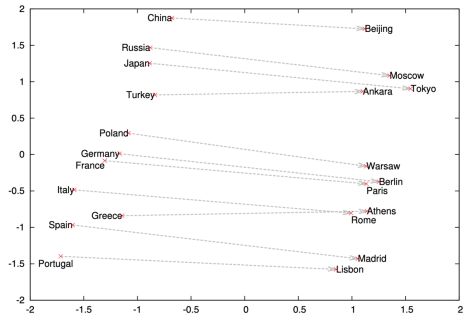
- Interessante Beobachtung: Wörter mit ähnlichen Bedeutungen tauchen in ähnlichen Kontexten auf
- Wörter werden Vektoren in $\mathbb{R}^N$ bei *relativ* großen $N \in [500, 2000]$ zugewiesen
- Diese Zuweisung (*Projektion*) wird optimiert, sodass $d(w_1, w_2) \sim d(c_1, c_2)$, wobei c_i der lexikalische Kontext von w_i ist
- Semantisch ähnliche Wörter tauchen in ähnlichen Kontexten auf, d.h. wenn die Kontexte im Vektorraum nah beieinander liegen, sollten auch die Wörter nah beieinander liegen

Embeddings

Word2Vec

So können implizit Beziehungen zwischen Wörtern gelernt werden

$$\text{vec}(\textit{Berlin}) - \text{vec}(\textit{Germany}) + \text{vec}(\textit{France}) = ?$$



Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed Representations of Words and Phrases and Their Compositionality. *Advances in Neural Information Processing Systems, 26*

Embeddings



- Word-Embeddings können zum Beispiel für *Information Retrieval* verwendet werden, ohne dass per se ein Sprachmodell angewendet werde muss (z.B. in eigentlich jeder digitalen Schlagwortsuche, oder um in RAG-Systemen Sprachmodelle mit dem richtigen Kontext zu versorgen)
- Word-Embeddings erleichtern das Training vom *eigentlichen* Sprachmodell (einfacheres Trainingsziel, wiederverwenden von bestehenden Modellen), oder werden effektiv beim Lernen von großen Modellen mitgelernt



§4

Neuronale Sprachmodelle

Definition

DEF 2.1 · Probabilistisches Sprachmodell

Ein **statistisches** oder **probabilistisches Sprachmodell** kann durch die *bedingte Wahrscheinlichkeit* des nächsten Wortes, gegeben aller vorherigen Worte repräsentiert werden. Dann ist die Wahrscheinlichkeit einer Wortfolge

$$\hat{P}(w_1, \ldots, w_T) = \prod_{i=1}^{T} \hat{P}(w_i \mid w_1, \ldots, w_{i-1}).$$

DEF 4.1 · Neuronales Sprachmodell

f sei ein neuronales Netz, parametrisiert durch θ, und w_1^T eine Wortfolge der Länge T. Dann ist

$$\hat{P}(w_T \mid w_1^{T-1}) = f(w_1^{T-1}, \theta)$$

ein **neuronales Sprachmodell**.

Attention

Motivation

- Frühe neuronale Sprachmodelle konnten zwar Informationen über längere Sequenzen verarbeiten als N-Gram Modelle, litten aber unter einem graduellen Verlust von Kontextinformationen
- Solche *Rekurrente Neuronale Netze* geben Informationen aus vergangenen Netzwerkdurchläufen *direkt verkettet* an spätere Durchläufe weiter
- In solchen Netzwerken ist es besonders schwierig, Informationen aufgrund ihrer *Relevanz*, anstelle ihrer Position, zu verarbeiten und über längere Sequenzen hinweg zu priorisieren

Attention



- Aus jedem Token in einer Folge von Token-Embeddings wird mit gelernten Projektionen ein *Query*, *Key*, und *Value* Vektor erstellt

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Die softmax-Verteilung sorgt dafür, das je nach Höhe des Skalarproduktes zwischen *Query* und *Key* ein entsprechender Anteil des zum *Key* gehörenden *Value* in das Ergebnis einfließt (der $\frac{1}{\sqrt{d_k}}$ Skalar sorgt für eine stabile Verteilung auch bei hohen Dimensionen)
- Nach einer Attention-Schicht beinhaltet die Sequenz also nicht nur Informationen zur Semantik der einzelnen Tokens (durch Embeddings), sondern auch Informationen zu deren Kontext

EXKURS · Analogie: Hashmaps

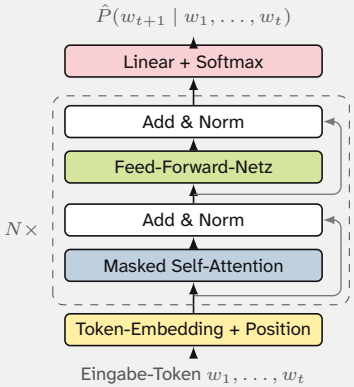
Hashmaps, oder Hash-Tabellen, sind eine beliebte Datenstruktur, die flexiblen und äußerst schnellen Zugriff auf ihre gespeicherten Daten bieten (Komplexität $O(1)$).

In der Anwendung wird über Schlüssel (*Keys*) auf die dahinter liegenden Werte (*Values*) zugegriffen.

Eine Anfrage (*Query*) in dieser Analogie würde nicht perfekt auf einen Schlüssel passen, sondern müsste zunächst bei allen Schlüssel abgleichen, wie gut sie zur Anfrage passen.

Transformer

- Die Abbildung rechts zeigt einen „*Decoder-Only-Transformer*“, die Standard-Architektur für moderne große Sprachmodelle
- „+ Position“: Transformer nutzen keine Rekurrenz (d.h. weitergabe von vorherigen Netzwerkläufen), und die reinen Matrizenmultiplikationen aus den Schichten des Netzwerks sind invariant zur Reihenfolge der Tokens. Deshalb benötigen die Token-Embeddings eine additive Komponente, aus der die Reihenfolge der Tokens nachvollziehbar wird.
- Am Ende wird eine Softmax-Verteilung über das Vokabular aufgespannt.
  $\rightarrow$ Die Auswahl des nächsten Tokens entspricht also effektiv einer *Klassifikation*.



Autoregression

Sprachgenerierung mit neuronalen Sprachmodellen ist Klassifikation.

- Das Modell gibt eine Wahrscheinlichkeitsverteilung über das Vokabular aus (d.h. „Wie wahrscheinlich ist es, dass *dieses* Token das nächste in der Sequenz ist?“)
- Das nächste Token wird entweder deterministisch gewählt, indem das wahrscheinlichste Token gewählt wird, oder indem gewichtet aus der Verteilung „gezogen“ wird.

DEF 4.2 · Autoregression

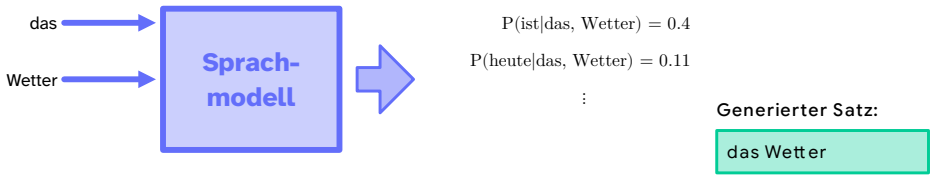
- Transformer generieren Text Schritt-für-Schritt
- In jedem Schritt wird das generierte Token zur Kontextfolge hinzugefügt, um die Generierung des nächsten Tokens zu beeinflussen
  → **Autoregression**
- Jede Position kann sich bei der Berechnung der *Attention* nur auf vorherige Positionen in der Folge beziehen (*kausale Maske*)

Sprachgenerierung

Beispiel

Sprachgenerierung:

- Ziehe das nächste Wort aus der Ausgabe (gewichtete Wahrscheinlichkeitsverteilung) des Sprachmodells
- Füge das gezogene Wort iterativ zur Bedingung hinzu

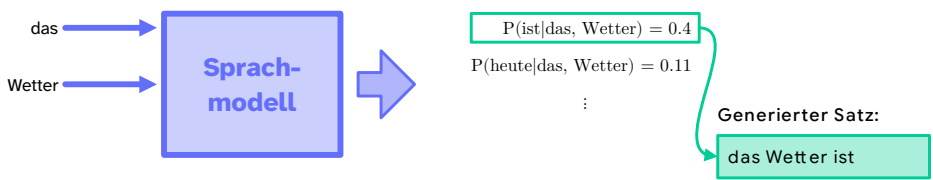


Sprachgenerierung

Beispiel

Sprachgenerierung:

- Ziehe das nächste Wort aus der Ausgabe (gewichtete Wahrscheinlichkeitsverteilung) des Sprachmodells
- Füge das gezogene Wort iterativ zur Bedingung hinzu



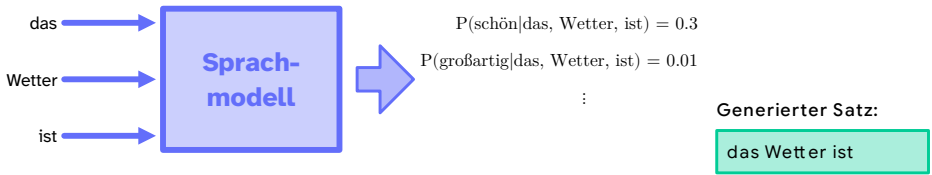
Sprachgenerierung



Beispiel

Sprachgenerierung:

- Ziehe das nächste Wort aus der Ausgabe (gewichtete Wahrscheinlichkeitsverteilung) des Sprachmodells
- Füge das gezogene Wort iterativ zur Bedingung hinzu

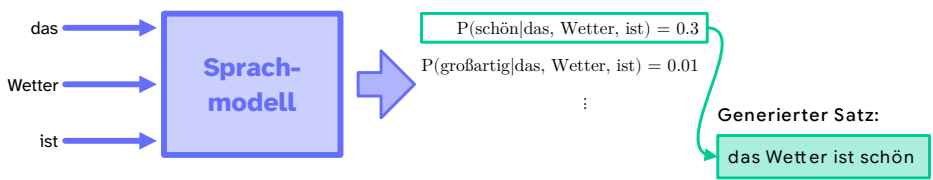


Sprachgenerierung

Beispiel

Sprachgenerierung:

- Ziehe das nächste Wort aus der Ausgabe (gewichtete Wahrscheinlichkeitsverteilung) des Sprachmodells
- Füge das gezogene Wort iterativ zur Bedingung hinzu





Was braucht ein Sprachmodell, damit es zum Chat-Modell wird?

Dialogsysteme

Vom Sprachmodell zum Chat-Interface

- **Antworten auf Fragen:** Anstelle von „frei zu assoziieren“, sollte das Modell tatsächlich auf Fragen antworten können
- **Langer Kontext:** Das Modell sollte in der Lage sein, lange Gesprächsverläufe als Kontext für die Generierung von Antworten zu Rate zu ziehen. Nutzer:innen sollten das Modell nicht alle drei Nachrichten an wichtige Informationen „erinnern“ müssen
- **Geschwindigkeit:** Das Modell sollte möglichst schnell Antworten geben können. Da Grafikprozessoren (GPUs) explizit dafür gebaut sind, Matrixmultiplikationen zu rechnen, können neuronale Sprachmodelle gut mit Rechenleistung skalieren

→ Erst mit Transformern konnten Sprachmodelle so mächtig werden dass Anwendungen wie ChatGPT möglich wurden

Zusammenfassung



§1 Grundlagen

Regelbasiert (ELIZA) → statistisch

Maschinelles Lernen: Leistung steigt mit Erfahrung

Neuronale Netze: differenzierbare Funktionen mit lernbaren Gewichten

Grenze: Regeln skalieren nicht auf die Vielfalt natürlicher Sprache

§2 N-Gramme

Sprache als Wahrscheinlichkeit: $\hat{P}(w_i \mid w_1, \ldots, w_{i-1})$

Markov-Annahme: nur die letzten $N-1$ Token zählen

Training durch Zählen, Generierung durch „Würfeln“

Grenze: $|V|^N$ wächst exponentiell, Gedächtnis nur $N-1$ Token

§3 Embeddings

Token werden zu reellwertigen Vektoren

One-Hot: simpel, aber riesig und ohne Semantik

Word2Vec: ähnliche Bedeutung → ähnliche Vektoren

Grenze: Ein Wortvektor allein kennt seinen Kontext nicht

§4 Transformer

Attention: Kontext nach *Relevanz* statt Position

Transformer: parallel auf GPUs, skalierbar

Autoregression: nächstes Token wählen = Klassifikation

Ergebnis: Grundlage für Chat-Modelle wie ChatGPT

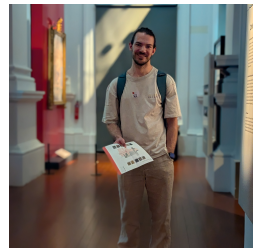
Kernidee

Ein Sprachmodell sagt immer nur das *nächste Token* vorher.

Über Uns



erik.voigt@uni-bamberg.de



johannes.langer@uni-bamberg.de

Wissenschaftliche Mitarbeiter am Lehrstuhl Kognitive Systeme der Universität Bamberg, Forschung und Lehre im Bereich KI und maschinelles Lernen



§6

Backup Slides

Skalierte Dot-Product-Attention

Herleitung $\frac{1}{\sqrt{d_K}}$

Annahme: Q und K sind unabhängig, mit $\mu = 0$ und $\sigma^2 = 1$

$$\begin{aligned}
\mathrm{Var}(S_{ij}) &= \mathbb{E}\left[\left(\sum_l Q_{il}K_{jl}\right)^2\right] && \text{(da } \mathbb{E}[S_{ij}] = 0\text{)} \\
&= \sum_l \mathbb{E}[Q_{il}^2]\,\mathbb{E}[K_{jl}^2] + \sum_{l\neq r} \underbrace{\mathbb{E}[Q_{il}]\mathbb{E}[K_{jl}]\mathbb{E}[Q_{ir}]\mathbb{E}[K_{jr}]}_{=0} && \text{(statistische Unabhängigkeit)} \\
&= \sum_{l=1}^{d_k} 1 \cdot 1 = d_k
\end{aligned}$$